

Ben Ford, Braxton Walk, Logan Bassett

Sam Micka

CS 470

December 10, 2025

CART Decision Tree Algorithm

Intro:

Decision tree learning refers to a class of supervised learning algorithms that use recursive partitioning to predict an outcome by repeatedly dividing the feature space into more homogeneous subsets. These algorithms follow a greedy, top-down strategy: at each step, they choose the split that best improves the purity of the resulting partitions before recursing on each side. By default, most decision tree algorithms are greedy algorithms, that is they make a greedy selection at every junction which leads to an overall greedy solution to the problem. They are also a popular choice of algorithm for their intuitive tree-like structure while still providing an optimized framework.

Among decision tree methods, the Classification and Regression Tree (CART) algorithm has become the standard formulation. The CART algorithm handles both classification and regression. CART constructs a binary tree and provides a unified framework for both classification and regression, allowing it to handle continuous and categorical features without prior discretization. Its generality, clear splitting criteria, and practical performance have made it a foundational component of modern ensemble methods such as Random Forests and Gradient Boosting.

Historical development of decision-tree algorithms traces back to two major lines of work in the late 1970s and early 1980s. J. Ross Quinlan introduced the ID3 algorithm (Iterative

Dichotoniser 3), which focused solely on classification and required categorical inputs.

Independently, Breiman, Friedman, Olshen, and Stone from Stanford University and California Berkley introduced CART, which addressed many of ID3's limitations by supporting continuous features, pruning methods, and regression. Because CART is the more general and widely adopted framework, this paper focuses on CART while briefly contrasting it with ID3 where relevant.

Problem Statement:

The CART algorithm addresses the problem of learning a predictive model from labeled data by constructing a binary decision tree. Formally, the input consists of a dataset $X \in R^{n \times d}$ containing n observations and d features, along with a corresponding target variable y . The target may be categorical, in which case the task is classification, or continuous, in which case the task is regression. The goal is to partition the feature space into increasingly homogeneous regions by recursively selecting split points that locally optimize a chosen impurity measure.

CART uses a greedy, top-down approach. At each node, the algorithm evaluates all possible feature thresholds and selects the split that yields the greatest reduction in impurity for classification or the greatest reduction in variance for regression. This process continues until stopping criteria are met, producing a decision tree whose leaf nodes contain class labels or predicted numerical values. The output of the algorithm is this tree, which can be used to make predictions on new data by traversing the splits from the root to a leaf.

Applications:

Decision trees show up in a lot of real-world systems because they are fast, easy to interpret, handle non-linear data, and work well as part of ensembles like Random Forests. One of

their biggest advantages is that you can actually follow the logic of the model. Each split represents a simple decision, so you can trace exactly how the final prediction was made. This interpretability matters in fields where people need to understand why a model returned a certain result. In finance, decision trees are used for credit risk scoring and for spotting unusual transactions that might be fraudulent, as well as predicting likelihood of someone paying back their loan. In healthcare, they help with disease prediction and basic diagnostic decision-making, giving medical staff a clear explanation of how the model reached its output and aiding in creating a diagnosis. Companies use them in marketing and sales for recommendations and ad targeting, picking the most relevant ad for a particular customer. Engineering teams rely on them for things like quality control and predicting when equipment might fail. These are just a few of the many industries where decision trees are prominent tools for prediction. Decision trees are also important building blocks for many other ensemble methods, which use multiple trees together to boost accuracy and stability. All of these examples explain why decision trees remain a common and reliable option for prediction across many different industries.

Recent Work:

<https://www.sciencedirect.com/science/article/pii/S014036642200425X>

CART decision trees have recently been applied to resource allocation in Wireless Sensor Networks (WSNs), where the goal is to determine how much bandwidth each Cluster Head (CH) should receive. A CH is an Internet-of-Everything (IoE) device that collects and forwards sensor data, so efficient bandwidth allocation is essential for maintaining network performance.

In this application, the CART model is trained using several network and environmental features, including:

- Distance to the base station
- Degree of connectivity (number of neighboring nodes)
- Link quality
- Congestion level
- Data size collected by the CH
- Data-forwarding capacity
- Overall channel conditions

Using these inputs, the CART model predicts the optimal bandwidth allocation category for each CH. Compared to logistic regression and neural-network-based approaches, CART achieved higher accuracy and more consistent performance, particularly under fluctuating network conditions. This improved prediction capability led to reduced transmission delay, more efficient bandwidth utilization, and higher packet-delivery ratios across the WSN.

<https://link.springer.com/article/10.1007/s40899-022-00776-0>

Another recent application of CART decision trees is in predicting water quality parameters, specifically dissolved oxygen and phosphorus levels in Lake Erie. The data included 15 years of monitoring data from 2006-2021. The study used several decision-tree-based models, CART, CHAID, QUEST, and C5, to determine how accurately each could forecast numerical water quality values.

Each model uses different splitting strategies and training procedures. CART and CHAID are primarily used for predicting continuous values, while QUEST and C5 classify measurements into categorical ranges. CART outperformed CHAID in predicting the dissolved oxygen and

phosphorus values, showing higher accuracy. For category-range prediction, C5 outperformed QUEST, achieving better classification accuracy.

Overall, the experiment showed that decision trees, specifically CART, are effective for water-quality prediction because they handle missing data gracefully, capture non-linear relationships among chemical variables, and automatically identify the most influential factors, such as temperature and ammonia concentration.

<https://www.sciencedirect.com/science/article/pii/S0377221715004208>

Another application of CART is in credit scoring, where models are used to predict whether a loan applicant is likely to repay a loan. This problem is important for the financial world such as institutions and banks, which must distinguish between low-risk and high-risk borrowers. In a large-scale benchmarking study, CART was evaluated alongside other state-of-the-art classification algorithms using publicly available financial datasets. These datasets contain features commonly used in credit evaluation, such as income, credit history, and debt-related variables.

CART and other decision-tree methods are particularly valuable in credit scoring because they generate interpretable, rule-based decision structures. Clear and human readable rules help loan officers understand why a prediction was made. This interpretability, combined with competitive predictive performance, makes CART a strong candidate for credit-risk assessment.

Many firms, corporations and governments use CART to classify data. We can see the diverse use of this algorithm by the examples above and how useful it is. There are many other

fields that use Decision Trees which shows its diverse utility and ability to be adapted to many environments.

Algorithmic Solution:

The CART algorithm tries to find splits that make the child nodes as “pure” as possible. Purity means that the samples in a node mostly belong to the same class. To measure how pure or impure a node is, CART uses the Gini impurity. The Gini impurity is a range between one and zero, one being a perfectly pure node with all samples belonging to one class, and zero a perfect impurity where all classes are equally distributed. This value approaches one with more classes, where two evenly split classes gives a Gini impurity of 0.5, while 4 classes gives a value of 0.75. This value is found with the equation $1-(1/K)$, where K is the number of classes.

Pseudocode:

1. Compute $G(T)$
2. For each attribute A in T:
3. For each possible split s of A:
4. Split T into subsets $T_1, T_2, \dots, T_k$
5. Compute $G_{split}(A, s)$ as:

$$G_{split} = 0$$

For each subset T_i :

$$G_{split} = G_{split} + \left(\frac{|T_i|}{|T|}\right) * G(T_i)$$

6. Compute $Gain = G(T) - G_{split}$

7. Select the attribute and split (A^* , s^*) with the largest Gain
8. Create a decision node using split (A^* , s^*)
9. For each subset T_i from (A^* , s^*):
10. If T_i is pure or empty:
11. Make a leaf node
12. Else:
13. Recursively repeat steps 1–5 on T_i

(Line 1) To minimize Gini impurity, the Gini impurity for the entire dataset is calculated first.

This gives a baseline measure for how mixed the data is before any splitting. Below is the formula for calculating the Gini Impurity:

$$Gini_Index(T) = 1 - \sum_{i=1}^m P_i^2$$

Here, T is the dataset that is being evaluated, m is the number of classes, and P_i is the probability in class i .

(Lines 2-6) For each attribute, CART tries all possible split points. Each split divides the data into subsets $T_1, T_2, \dots, T_k$. The Gini impurity is computed for each subset using the same formula. A weighted average of these subset Gini values becomes the impurity of that split.

(Line 6) The algorithm then calculates the “gain” for that split. This is the reduction in impurity:

$$Gain = G(T) - G_{split}$$

(Line 7) The attribute and split with the highest gain are chosen. This becomes the next decision node.

(Lines 9-13) The process is repeated recursively on each child subset. If a subset is pure (Gini = 0) or empty, that branch becomes a leaf node. Otherwise, CART keeps splitting until stopping conditions are met.

Example video here:

<https://www.youtube.com/watch?v=xyDv3DLYjfM>

Time Complexity: Suppose we have n = number of samples, and d = number of attributes (features). To find the best split for a single attribute, the algorithm must: 1) Sort the values of that attribute (cost: $O(n \log n)$) Evaluate all possible split points (cost: $O(n)$). This gives $O(n \log n)$ per attribute. Because CART does this for all d attributes, the cost of finding the best split at a single node is: $O(d * n \log n)$.

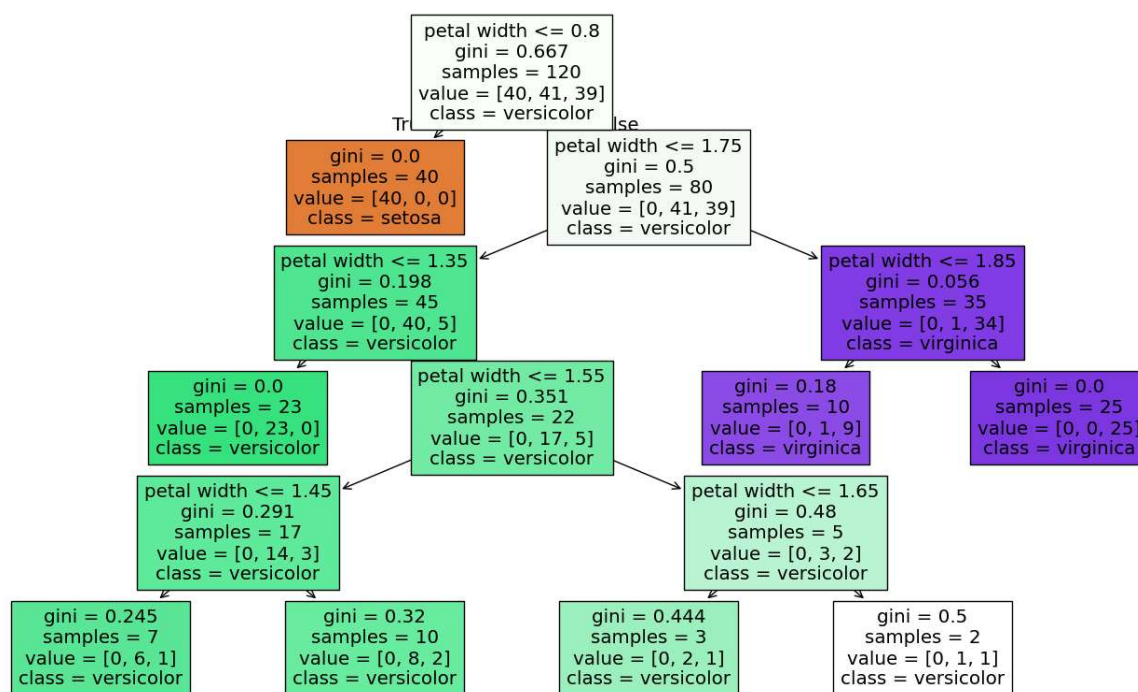
After choosing the best split, the dataset is divided into two smaller subsets. The same process repeats recursively on each subset until reaching leaf nodes. In the worst case, the tree becomes highly unbalanced (similar to a linked list), and the algorithm processes nearly all elements at each level. This produces a worst-case overall training complexity of: $O(d * n^2 \log n)$. In practice however, the trees are usually more balanced resulting in a time complexity of $O(d * n \log^2 n)$.

Summary:

- Best split at one node: $O(d * n \log n)$
- Worst-case full tree: $O(d * n^2 \log n)$
- Average-case full tree (balanced): $O(d * n \log^2 n)$

Example: A CART Decision Tree was applied to a small data set to predict which class an iris flower falls into. We have 3 classes {Versicolor, Setosa, Virginica} and 120 datapoints. For this example we are only using the Petal Width as our attribute **input** to keep the tree small and simple as trees can grow at a fast rate with more features. Our **output** will be the predicted iris class.

Below is the picture of the tree. Under said picture we will discuss the operations the tree took to come to its final predictions.



At the root of the tree we see a gini impurity of 0.667 since there are 3 classes to choose from $1 - 1/3 = 0.667$. The left leaf becomes true and the right becomes false. We see at the start that the left leaf is finished as it has a gini value of 0.0 and instantly classifies the iris as a Setosa because its petal width ≤ 8.0 . If the petal width > 8.00 then it moves to the right leaf. A new gini value is then computed by the new attribute value that best splits up the data. The

process is repeated recursively on each leaf subset. If a subset is pure ($\text{Gini} = 0$) or empty, that branch becomes a leaf node and ends. Otherwise, CART keeps splitting until stopping conditions are met or no more splitting can be done.

Algorithmic Discussion:

The CART algorithm is very flexible for a decision tree model, but it still requires a few set assumptions about its inputs. To have a stable tree, classes must be evenly distributed. CART has a tendency to favor the majority class when producing trees, which can lead to biased outcomes. The decision tree is sensitive to all changes in data, leading to drastically different trees from small changes, making the algorithm unstable for many purposes. The tree can often become overfit, especially with large, shallow class sizes.

To overcome this, the CART algorithm, like many other decision tree algorithms, uses tree pruning to help prevent overfitting of the data. CART also uses other techniques to create better trees for prediction. Methods like recursive partitioning repeatedly search for the best split point for each feature and select the one that results in the most homogeneous child nodes.

<https://medium.com/@denizgunay/cart-4838fec3e405>

An alternative algorithm is the ID3 algorithm, which is a faster option than CART and more stable, but ID3 struggles to handle continuous data, and has greater problems with overfitting, especially with smaller data sets.

https://ijirem.org/DOC/12_performance-analysis-of-cart-and-id3-algorithm-for-heart-prediction-system.pdf

A potential change to the CART algorithm could be to use an algorithm that isn't greedy to look ahead in the potential tree. This would allow for better accuracy when creating the tree, but the time complexity would increase exponentially.

While the fundamentals of Decision Trees are often based off of CART it is not the industry standard. **Gradient-boosted tree ensembles** lead the way for practical applications and data science competitions. As these models allow each new tree to correct the errors of the previous one. Allowing for a more accurate tree rather than CARTS one time generation of a tree.